

Universidad Nacional de Luján

Trabajo Práctico 02: Procesos ETL

Alumno: Huici Nicolas

Legajo: 165755

1. Introducción

En este trabajo práctico armamos procesos ETL (Extracción, Transformación y Carga) sobre el dataset de Education Statistics del Banco Mundial. La idea era resolver los mismos ejercicios de dos formas distintas y después compararlas.

Las dos formas fueron: (1) escribiendo código con **Pandas** en una notebook de Jupyter, y (2) armando flujos visuales con **Apache Hop**.

Teníamos que generar tres archivos CSV: uno de países, uno de preguntas (indicadores) y uno con la duración de la educación primaria en 2023. Después había que contestar cuatro preguntas comparando las dos herramientas. Todo lo que sigue está explicado desde **mi experiencia** al hacerlo, siendo principiante en las dos herramientas.

2. El dataset

El archivo que usamos es `API_4_DS2_en_csv_v2_3434744.csv`, el de Education Statistics del Banco Mundial. Sus características principales:

- Las primeras 4 columnas son `Country Name`, `Country Code`, `Indicator Name` e `Indicator Code`. Después hay una columna por año, de 1960 a 2023.
- Cada fila es un país con un indicador, y los años van en columnas (formato "ancho").
- Hay **266 países o entidades** y **164 indicadores** distintos. Cada país se repite una vez por indicador.
- Detalles técnicos: al final de cada línea hay una coma de más, así que aparece una columna vacía (`Unnamed: 68`) que se descarta. Además, el archivo empieza con un carácter invisible (BOM), por lo que en Pandas se lee con `encoding="utf-8-sig"`.

Decisiones que tomé

Decisión 1: dejar los 266 registros de países, incluidos los agregados regionales. En el dataset, además de países reales, hay entidades como "World", "High income" o "Euro area". Las dejé porque cada una tiene nombre y código únicos, y porque el enunciado habla de "país o entidad regional".

Decisión 2: descartar los países sin dato en 2023 en la consigna 3. El indicador de duración de la primaria aparece 266 veces (una por país), pero 11 no tienen dato en 2023. Elegí descartarlos (dropna) por estas razones:

- Diez de los once no tienen ningún dato en ningún año de la serie (1960 a 2023), así que no hay un valor anterior del cual copiar.
- Inventar un valor (por ejemplo con la mediana o el valor más común, que es 6 años) no tiene sentido: la duración de la primaria es un dato de cada país, no una medición que uno pueda estimar.
- La única excepción es Afghanistan, que tiene datos hasta 2022. Al descartarlo se pierde ese país, pero la consigna pide específicamente la medición de 2023.

Con esto quedan **255 países** en el CSV de la consigna 3.

Decisión 3: desempate en el orden de la consigna 3. La consigna pide ordenar de mayor a menor duración, pero hay muchos empates (166 países tienen 6 años). Para que el orden quede fijo, dentro de cada duración ordené por código de país de la A a la Z.

3. Resolución de las tres consignas

Consigna 1: CSV de países

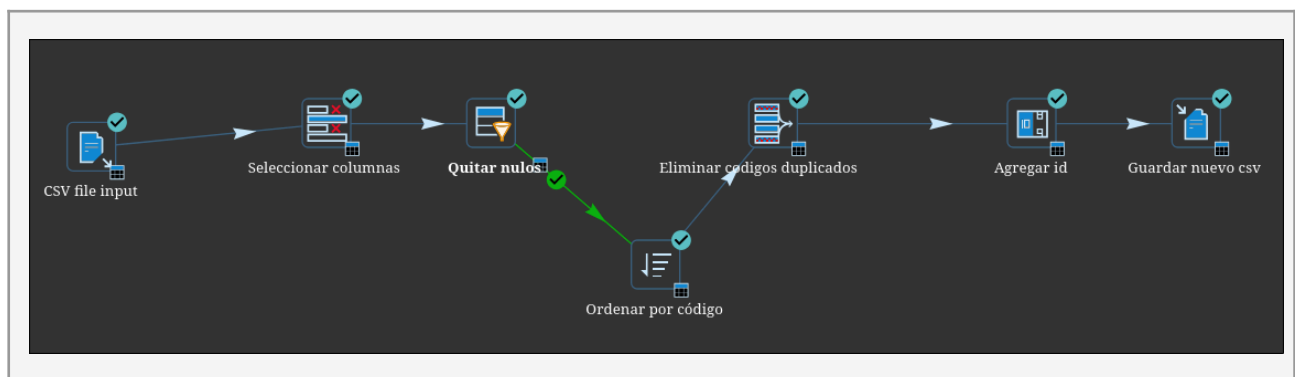
En Pandas

Me quedé solo con las columnas `Country Name` y `Country Code`, las renombré a `nombre_pais` y `codigo_pais`, saqué nulos y duplicados, y creé el `id` de 1 en adelante. El orden de las operaciones importa.

En Apache Hop

El pipeline tiene estos pasos: `CSV file input` > `Select values` > `Filter rows` > `Sort rows` > `Unique rows` > `Add sequence` > `Text file output`.

En Hop, `Unique rows` solo elimina repetidos que están uno al lado del otro, por eso hace falta el `Sort rows` antes.



Resultado

El CSV tiene **266 filas** (ids de 1 a 266), sin nulos, con las columnas `id`, `nombre_pais` y `codigo_pais`. Al contar líneas, el archivo de Hop mostraba 268: son el encabezado, los 266 registros y una línea vacía al final, que es solo el salto de línea de cierre del archivo y no un registro.

Consigna 2: CSV de preguntas

En Pandas

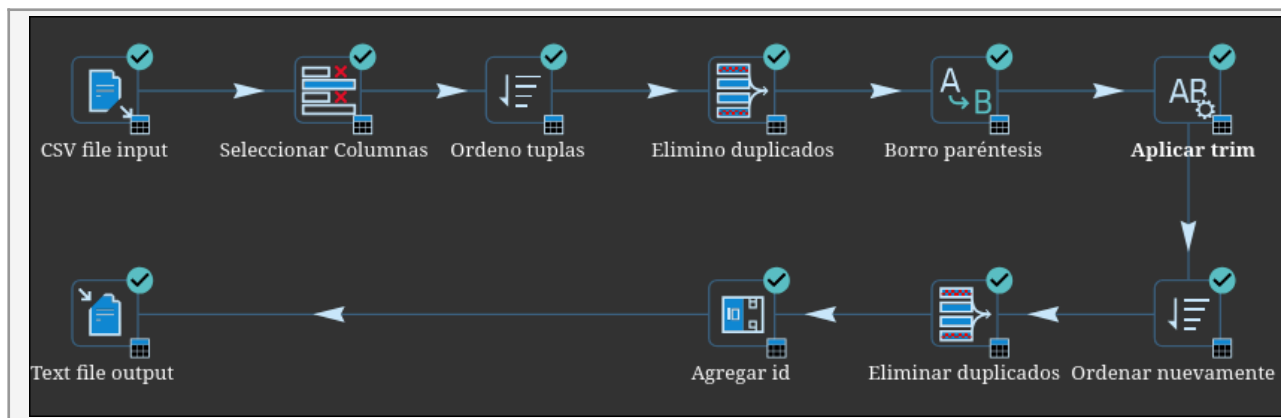
Tomé la columna `Indicator Name`, saqué repetidos, y con una expresión regular borré todo lo que está entre paréntesis (incluidos los paréntesis) y apliqué `strip()`. Después volví a sacar repetidos, porque dos indicadores distintos pueden quedar iguales una vez limpios. Así pasé de 164 a **151** preguntas únicas.

La expresión final fue `\s*\([^)]*\)`. Mi primera versión era `\([^)]*\)`, y al revisar el resultado vi que 5 preguntas quedaban con un espacio antes de la coma (por ejemplo `School enrollment, primary , gender parity index`). Lo arreglé agregando `\s*` al principio, que borra también los espacios previos al paréntesis. Además ordené las preguntas alfabéticamente sin distinguir mayúsculas de minúsculas, para que el `id` coincida con el de Hop.

En Apache Hop

`CSV file input` > `Select values` > `Sort rows` > `Unique rows` > `Replace in string` (con la opción de `Regex` activada) > `String operations` (`trim`) > `Sort rows` > `Unique rows` > `Add sequence` > `Text file output`.

Hicieron falta dos rondas de `Sort rows` con `Unique rows`: la segunda es sobre el texto ya limpio, y tiene que ordenarse por ese campo para que los repetidos queden juntos.



Resultado

El CSV tiene **151 filas**, con `id` de 1 a 151, sin paréntesis, sin espacios sobrantes y sin nulos.

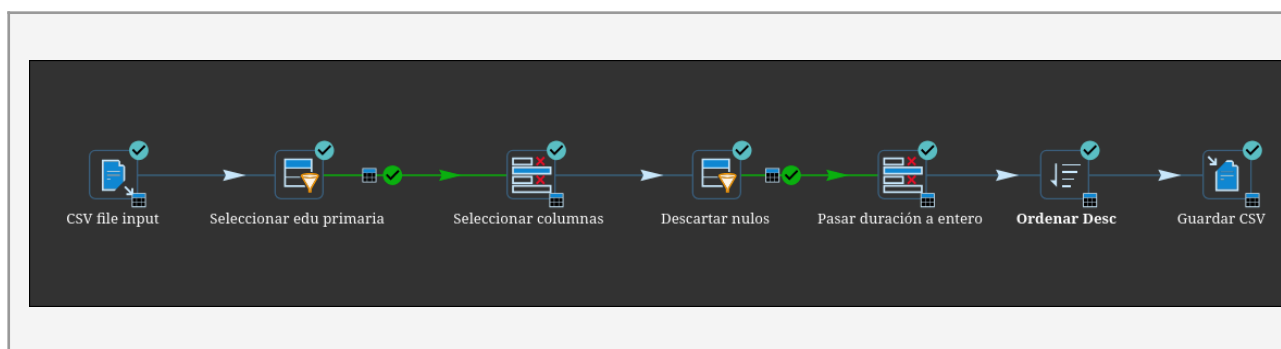
Consigna 3: educación primaria en años (2023)

En Pandas

Filtré el indicador de duración de la primaria por su código, `SE.PRM.DURS`, y no por el nombre. Esto es importante: existe un indicador muy parecido, `SE.PRE.DURS`, que es el de **preescolar**. Me quedé con las columnas de país, código y 2023, las renombré, descarté los nulos, pasé la duración a entero y ordené de mayor a menor duración (con desempate por código de país).

En Apache Hop

CSV file input > Filter rows (`código de indicador = SE.PRM.DURS`) > Select values > Filter rows (descartar nulos de 2023) > Select values (pasar a entero) > Sort rows > Text file output.



Resultado

El CSV tiene **255 filas**. Las dos primeras son Guam e Ireland, con 8 años, y la última es Uzbekistan, con 4 años.

Verificación: Pandas y Hop dieron lo mismo

Para comprobar que los dos enfoques generaron el mismo resultado, escribí una celda en la notebook que lee los CSV de Pandas y los de Hop y los compara (cantidad de filas, columnas, nulos y contenido). Esa verificación me sirvió para encontrar diferencias que Hop no me avisó: por ejemplo, el orden de las columnas de los CSV de países y primaria no coincidía con el de Pandas ni con el que pedía la consigna, y hubo que corregirlo en el último `Select values` de cada pipeline. También los `id` de las preguntas no coincidían hasta que igualé el criterio de ordenamiento en Pandas.

4. Respuestas a las preguntas

Pregunta 1: Facilidad de Desarrollo y Curva de Aprendizaje: ¿Cuál de los dos enfoques (Pandas vs. Apache Hop) resultó más intuitivo para realizar transformaciones de texto complejas como la limpieza mediante expresiones regulares (Regex)? Justifique evaluando el tiempo de desarrollo y la legibilidad de la solución.

Siendo nuevo en las dos herramientas (Pandas y Apache Hop), me resultó más intuitivo Apache Hop, incluso para la limpieza de texto con Regex. Igual, esto depende de la experiencia de cada persona.

Para sacar todo lo que está entre paréntesis usé la misma expresión en las dos herramientas: `\s*\([^)]*\)`. Lo que cambia es dónde se escribe. En Pandas va adentro de una línea de código: `.str.replace(r"\s*\([^)]*\)", "", regex=True)`. En Hop son dos pasos: `Replace in string`, con la opción "use Regex" tildada, y `String operations` para el trim. En Hop, dentro de cada paso los campos ya están definidos, y cuando un campo acepta pocas opciones hay un desplegable con esos valores. Una vez que sabía qué paso usar, saber cómo completarlo era más intuitivo.

Para Pandas dependí mucho de la IA para escribir el código. No creo haber podido hacer el TP01 ni el TP02 a tiempo sin ella, porque leer la documentación y buscar ejemplos me habría llevado demasiado tiempo. Con Hop la usé solo para asesorarme en cómo crear un proyecto nuevo y en qué herramienta usar para una tarea puntual, por ejemplo "qué usar para reemplazar texto con Regex".

En las dos herramientas tardé más o menos lo mismo. Con Pandas ya había tenido un primer contacto, así que pude empezar enseguida. Con Hop, al ser la primera vez, tuve que dedicar tiempo a familiarizarme con el entorno, pero después trabajé mucho más solo.

Me parece que en Hop cuesta bastante menos. Los dos entornos ya venían armados (el docker de Hop y la notebook de Jupyter), así que la diferencia está en lo que hay que aprender: en Pandas hay que aprender la librería y además Python, y en Hop no hay que programar. Para alguien que nunca programó, la diferencia se nota todavía más.

Hop muestra la solución como un gráfico, y estamos más acostumbrados a ver las cosas así, así que me parece un poco más legible a primera vista. Para alguien sin experiencia en programación, la diferencia es todavía mayor: es mucho más simple leer la solución en Hop que en Jupyter. Hay un detalle: la Regex en sí no es más fácil de leer en Hop. Es el mismo patrón, pero queda escondido adentro de la configuración de `Replace in string`, y para verlo hay que abrir el paso. En la notebook, en cambio, la Regex se ve directo en la celda, junto con el resto del código.

Esto puede cambiar según quién lo use. Un desarrollador de Python, o un analista que hace tiempo trabaja con Pandas, quizás prefiera la flexibilidad y rapidez de Pandas frente a un entorno gráfico.

Pregunta 2: Manejo de Errores, Traceabilidad y Depuración (Debugging): Al momento de inspeccionar los datos intermedios o detectar inconsistencias durante la ejecución, ¿qué diferencias encontró entre el flujo gráfico de Apache Hop (sus transformaciones/steps y preview) y el entorno interactivo de una Notebook de Python (Jupyter/Colab)?

En depuración y manejo de errores me sentí bastante más cómodo en Jupyter que en Apache Hop.

Fue mi primera vez con Hop y tuve varios errores que tuve que descifrar. Para eso usé la función "preview and debug the pipeline", que me dio información útil sobre las transformaciones que se le iban

haciendo a los datos. Podía ver cómo cambiaban paso a paso. Eso me sirvió para saber, ante un fallo, en qué paso había ocurrido, y después investigar esa acción para entender qué había hecho mal o qué había malentendido. Cuando el pipeline falla, Hop lo marca en el paso que falló (según la documentación, con un triángulo rojo arriba a la derecha y el mensaje de error al pasar el mouse, y el detalle completo queda en los logs). El mensaje que muestra es más amigable que el traceback que tira Python. Después de ejecutar, cada paso tiene además un ícono de tabla para ver los datos que produjo.

En Jupyter puedo inspeccionar los datos en cualquier punto del proceso. Un ejemplo es la última celda del notebook, donde verifiqué que los .csv generados por los dos procesos dieran el mismo resultado final. También es mucho más fácil agregar pasos de debug en medio del código: imprimir un dato, hacer un `.describe()`, `.info()` o `.head()` es una línea, casi instantánea de escribir, y se puede poner en cualquier lugar. Eso me ayudó a entender mejor los datos (cantidad de nulos, tipos de cada columna, etc.) y a diagnosticar cuando el código no se comportaba como esperaba.

Los errores que detecté en este trabajo los encontré con la notebook, no con Hop:

- Las 5 preguntas con un espacio antes de la coma las encontré con `str.contains(r"\s,").sum()`.
- La línea extra en el CSV de países de Hop (268 líneas para 266 registros más el encabezado) la resolví leyendo el archivo en Python, y era solo el salto de línea final.
- El orden de columnas distinto en los CSV de países y primaria lo detecté comparando con Pandas. Hop no me avisó nada.

Para validar el resultado de Hop terminé usando Jupyter.

Esta diferencia puede deberse a que no tengo experiencia con Apache Hop, pero sí tengo experiencia depurando código (aunque no sea Pandas). Además, Hop permite hacer preview y debug de un paso sin modificar el pipeline. Pero cada preview implica ejecutar el pipeline, mientras que en Jupyter puedo consultar los datos de una celda sin volver a correr todo.

Pregunta 3: Escalabilidad y Rendimiento en Procesamiento Masivo: Pensando en un escenario de volúmenes masivos de datos (Big Data) que superen la memoria RAM de un solo equipo, ¿cuáles son las limitaciones de pandas y cómo Apache Hop permite gestionar/escalar flujos ETL mediante procesamiento distribuido o en streaming?

Pandas es muy útil para analizar y procesar datos que se pueden manejar en una sola máquina. Apache Hop me parece más adecuado para armar procesos ETL que manejan grandes volúmenes de datos y que pueden necesitar infraestructura distribuida o procesamiento continuo.

La principal limitación es que, en general, los datos que se procesan tienen que cargarse en la memoria RAM del equipo. Si el conjunto de datos es muy grande y supera la memoria disponible, Pandas puede volverse muy lento o directamente no poder procesarlo. Además, el procesamiento se hace principalmente en un único equipo, así que no está pensado para distribuir fácilmente grandes cantidades de datos entre varias máquinas.

En este TP el archivo es chico: el CSV pesa unos 21 MB, un tamaño que una computadora común maneja en memoria sin problemas. Aun así, para la consigna 3 Pandas leyó las 43.624 filas y las 69 columnas del archivo, cuando solo necesitaba 255 países. Con un archivo mucho más grande eso puede traer problemas, tanto de espacio como de tiempo de procesamiento. Según tengo entendido

Pandas permite leer solo algunas columnas (`usecols`) y también leer el archivo por partes (`chunksize`), pero son trucos que hay que programar a mano.

Hop permite trabajar con flujos ETL de una forma más escalable. Los procesos se pueden diseñar para que los datos se procesen por etapas, sin cargar todo el conjunto en memoria al mismo tiempo. Incluso para un paso que necesita ver todas las filas, como ordenar, el transform `Sort rows` que usé en los pipelines tiene un tamaño de ordenamiento configurable y un umbral de memoria libre: cuando se acerca al límite empieza a guardar datos en archivos temporales en el disco.

Además, Hop puede usar distintos motores de ejecución. Según su documentación, hay configuraciones para el motor propio de Hop (local o remoto) y para motores de Apache Beam como Spark, Flink y Google Dataflow. Esto permite aprovechar procesamiento distribuido cuando se trabaja con tecnologías preparadas para Big Data. También puede trabajar con procesamiento por streaming, donde los datos se procesan a medida que llegan y no hace falta esperar a tener todo el conjunto: tiene transforms para consumir datos de Kafka, Kinesis y Google Pub/Sub, entre otros.

Por todo esto, Pandas sirve cuando los datos caben en una sola máquina, y Hop es la mejor opción cuando los volúmenes son grandes y puede hacer falta una infraestructura distribuida o procesamiento continuo. Igual, en este TP todo corrió en una sola máquina y con un archivo chico, así que no pude comprobar esto en la práctica. Además, no todos los transforms funcionan en todos los motores: `Sort rows`, que usé, figura como soportado solo por el motor propio de Hop, así que para escalar en serio habría que rediseñar los pipelines.

Pregunta 4: Portabilidad, Automatización y Despliegue en Producción: Analice el costo de mantenimiento y orquestación en producción de ambas opciones: ¿Qué ventajas ofrece la dockerización de Apache Hop frente a la ejecución e integración de scripts/notebooks de Python mediante herramientas de orquestación (ej. Apache Airflow, Cron, etc.)?

Python con Pandas ofrece mucha flexibilidad para programar los procesos, pero requiere gestionar por separado el código, el entorno y la orquestación. Apache Hop, con sus flujos y la posibilidad de usar Docker, permite centralizar el código y el entorno, y facilita la portabilidad y el mantenimiento de los procesos ETL en producción. Igual, la programación de las ejecuciones sigue haciéndose desde afuera, con herramientas como Airflow, igual que con Python.

Los procesos con Pandas normalmente se ejecutan mediante scripts o notebooks, y necesitan un entorno de Python con todas las librerías y dependencias. Nunca usé Apache Airflow ni Cron, pero entiendo que mantener estos scripts en producción implicaría mantener actualizados el código, el entorno de ejecución de Python y las herramientas para administrarlos. Entiendo que esto es posible, pero no sería igual de fácil de mantener a medida que crece que con Apache Hop. Si no se tiene cuidado, puede volverse confuso o engorroso, por ejemplo con un rejunte de scripts sueltos.

Hop permite empaquetar el entorno de ejecución con Docker. Eso facilita llevar el mismo proceso a distintos equipos o servidores sin instalar y configurar a mano todas las dependencias. El entorno queda más portable y menos dependiente de la máquina donde se ejecute, porque las dependencias y sus versiones vienen empaquetadas en la imagen. Hop tiene una imagen oficial (`apache/hop`), y según su documentación una forma de usarlo en producción es armar una imagen con el proyecto adentro y ejecutar los flujos con `hop-run`, disparados desde herramientas como Airflow o Kubernetes. Además, los flujos viven dentro del proyecto de Hop y no como scripts sueltos, lo que reduce el riesgo de terminar con un proyecto que nadie puede entender.

Docker no es exclusivo de Hop: Python también se puede empaquetar en Docker, pero hay que armar la imagen y mantener las dependencias a mano. Con Hop la imagen ya viene armada y el flujo está dentro del proyecto.

Levantar Hop fue rápido: entorno de Docker estaba listo y configurado, con su propio compose, y solo tuve que ejecutar `docker compose up` para tener el contenedor accesible en `localhost:8090`. Eso muestra una ventaja concreta de Docker, que es no tener que instalar ni configurar nada a mano. Ahora bien, como el entorno está aislado, Hop y Jupyter no compartían archivos, y para poder comparar los resultados tuve que copiar los CSV a mano de uno a otro. La portabilidad de Docker viene con ese costo: para compartir archivos con el exterior hay que configurarlo.

Python con Pandas sirve mucho para programar los procesos con flexibilidad, pero hay que administrar por separado el código, el entorno y la orquestación. Apache Hop, mediante sus flujos y Docker, permite centralizar el código y el entorno, y facilita la portabilidad y el mantenimiento de los ETL en producción.

5. Conclusión general

Las dos herramientas me permitieron llegar exactamente al mismo resultado en las tres consignas, así que la comparación no es sobre qué hace cada una, sino sobre cómo es trabajar con ellas.

Como principiante, Apache Hop me resultó más intuitivo y más fácil de leer, porque los flujos se ven en un gráfico y cada paso tiene sus campos definidos. Pandas, en cambio, me resultó más cómodo para depurar, porque puedo mirar los datos en cualquier punto con una línea. En escalabilidad y en despliegue, Hop parece la mejor opción para volúmenes grandes y para llevar los procesos a producción, aunque en este TP no llegué a probar el procesamiento distribuido.

Algo que aprendí durante el trabajo es que las dos herramientas pueden fallar sin avisar: en Pandas, un `dropna()` que no guarda su resultado, y en Hop, un `Unique rows` sin ordenar antes o un `Select values` con las columnas en otro orden. Por eso me resultó clave verificar siempre el resultado final comparando los dos CSV, y no confiar solo en que el proceso terminó sin errores.